Engenharia de Controle e Automação

MIC - Microcontroladores

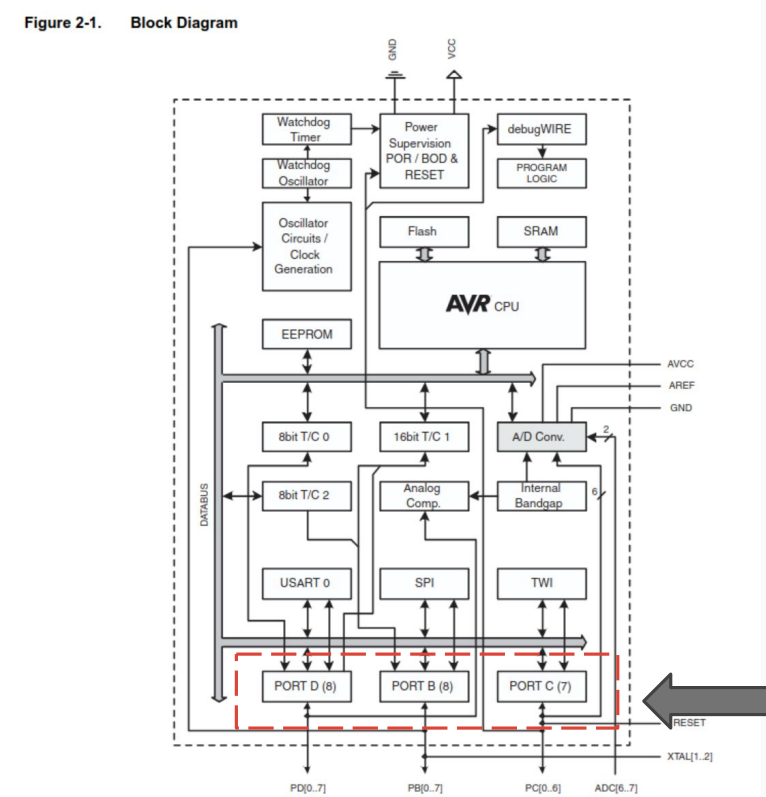
Periféricos AVR - GPIOs

Prof. Luiz Marangoni

Rev.2026.1

INSTITUTO FEDERAL
São Paulo
Câmpus Hortolândia

Diagrama em blocos do ATMega e GPIOs



GPIO - General Purpose I/O - Pinos de entrada/saída de uso geral

ATMega328 tem 03 conjuntos de GPIOs chamados PORTs

- PORTB - PB[0..7] - 8 pinos
- PORTC - PC[0..6] - 7 pinos
- PORTD - PD[0..7] - 8 pinos
- Total de **23 pinos**

- Todas as portas AVR possuem operação Read-Modify-Write (RMW) real.
- Isso permite alterar um único pino sem afetar os demais pinos da mesma porta.
- Isso é feito com instruções como SBI e CBI (Set Bit e Clear Bit)

No ATMega é possível, durante a execução do programa:

- Mudar a **direção** de um pino individualmente;
- alterar o **nível lógico de saída** de um pino configurado como saída;
- habilitar ou desabilitar o **resistor de pull-up** de um pino configurado como entrada.

Mapa de pinos ATMega328P - PDIP

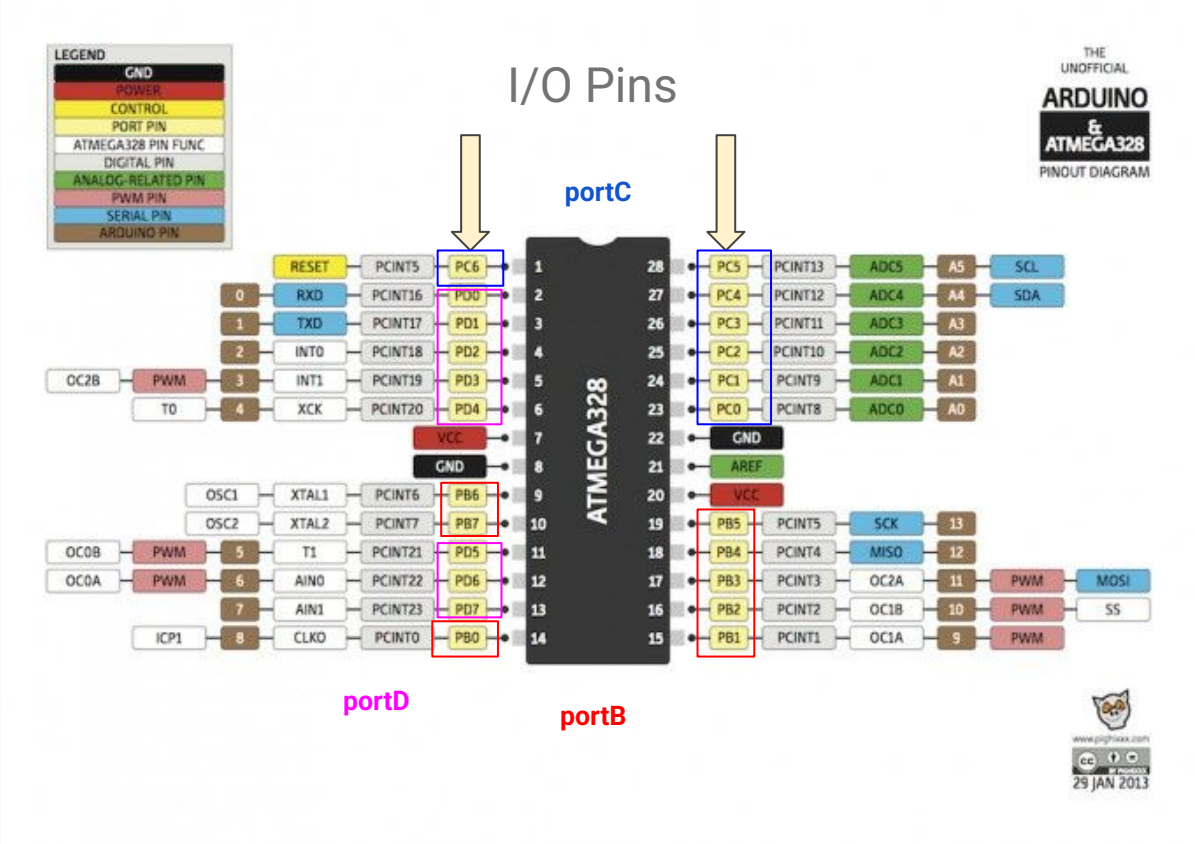
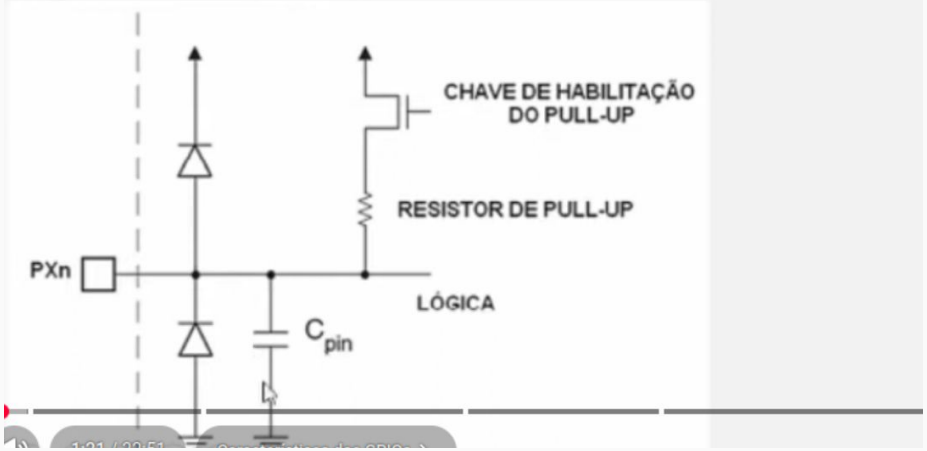


Diagrama e características elétricas do GPIOs

Cada pino pode ser ajustado individualmente

- Direção (Entrada ou Saída)
- Nível Lógico, se saída.
- Habilitação do resistor de Pull-up, se entrada.



Cada pino:

- Diodos de proteção
- Capacitor pequeno para filtragem sinais de altas frequências
- Resistor de pull-up que pode ou não ser habilitado para a entrada através de um bit de controle

Corrente máxima

- **De cada pino:** 20 mA
- **De cada porta:** 100 mA
- **Total do micro:** 200 mA

Registradores de I/O

- **DDRx**: registrador de direção, usado para definir se os pinos da porta X são entradas ou saídas.
- **PORTx**: registrador de dados, usado para escrever nos pinos da porta X, caso saída ou habilitar os resistores de Pull-up, caso entrada.
- **PINx**: registrador de entrada, usado para ler o conteúdo dos pinos do PORTx.
- **MCUCR:** o bit 4 desse registrador (PUD - Pull-up Disable) habilita ou desabilita todos os resistores de Pull-up

Table 14-1. **Port Pin Configurations**

DDRxn	PORTxn	PUD	I/O	Pull-up Hab?	Comentário
0	0	X	Entrada	Não	Alta impedância
0	1	0	Entrada	Sim	Entrada em 1 caso NC
0	1	1	Entrada	Não	Alta impedância
1	0	X	Saída	Não	Saída em 0
1	1	x	Saída	Não	Saída em 1

Exemplo 01 - Acionamento de um pino como saída

Algoritmo

1. **Escolher o pino** que será usado.
2. **Identificar a porta** à qual esse pino pertence
   Ex.: PB0, PB1, PD2, PC3.
3. **Configurar o pino como saída** no registrador de direção da porta
   - colocar o bit correspondente em `DDRx` com valor **1**.
4. **Definir o nível lógico desejado** no registrador de dados da porta:
   - escrever **1** em `PORTx` para colocar o pino em **nível alto**;
   - escrever **0** em `PORTx` para colocar o pino em **nível baixo**.
5. **Manter ou alternar o valor** do pino conforme a necessidade do programa.

Exemplo 01 - Acionamento de um pino como saída

Algoritmo

1. **Escolher o pino** que será usado.
2. **Identificar a porta** à qual esse pino pertence
   Ex.: PB0, PB1, PD2, PC3.
3. **Configurar o pino como saída** no registrador de direção da porta
   - colocar o bit correspondente em `DDRx` com valor **1**.
4. **Definir o nível lógico desejado** no registrador de dados da porta:
   - escrever **1** em `PORTx` para colocar o pino em **nível alto**;
   - escrever **0** em `PORTx` para colocar o pino em **nível baixo**.
5. **Manter ou alternar o valor** do pino conforme a necessidade do programa.

Exemplo:

Para o pino **PB5**:

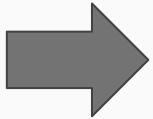
1. Selecionar o pino **PB5**.
2. Configurar **PB5** como saída em `DDRB`.
3. Escrever **1** em `PORTB` no bit 5 para ligar o pino.
4. Escrever **0** em `PORTB` no bit 5 para desligar o pino.

Exemplo 01 - Acionamento de um pino como saída

Exemplo:

Para o pino **PB5**:

1. Selecionar o pino **PB5**.
2. Configurar **PB5** como saída em DDRB.
3. Escrever **1** em PORTB no bit 5 para ligar o pino.
4. Escrever **0** em PORTB no bit 5 para desligar o pino.



Em C:

```
#include <avr/io.h>

int main(void)
{
        // Configura o pino PB5 como saída
        DDRB |= (1 << PB5);

        // Liga o pino PB5
        PORTB |= (1 << PB5);

        // Desliga o pino PB5
        PORTB &= ~(1 << PB5);

        while (1)
        {
        // loop infinito
        }
}
```

Exemplo 01 - Acionamento de um pino como saída

Operações Bitwise utilizadas

```
DDRB |= (1 << PB5);
```

Estou **setando** o pino PB5 do registrador DDRB - Configurando como **saída**

Estou **setando** o pino PB5 do registrador PORTB - Enviando um sinal **ALTO**

```
PORTB &= ~(1 << PB5);
```

Estou **resetando** o pino PB5 do registrador PORTB - Enviando um sinal **BAIXO**

De forma genérica

Para setar um bit:

```
REGISTRADOR |= (1 << BIT);
```

Para resetar um bit:

```
REGISTRADOR &= ~(1 << BIT);
```

Significado

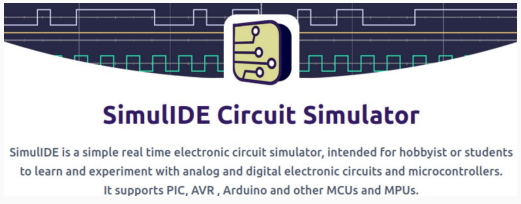
(1 << BIT) - Cria uma máscara com 1 apenas no bit desejado, demais com 0

~(1 << BIT) - Cria uma máscara com 0 apenas no bit desejado, demais com 1

|= Operação **OR bit a bit** para ligar apenas o bit desejado sem alterar os demais

&= Operação **AND bit a bit** para desligar apenas o bit desejado sem alterar os demais

Simulação 1 - Saída



SimulIDE

Exemplo 02 - Acionamento de um pino como entrada

Algoritmo

Algoritmo

1. **Escolher o pino** que será usado.
2. **Identificar a porta** do pino
   Ex.: `PB0`, `PC2`, `PD7`.
3. **Configurar o pino como entrada** no registrador de direção:
   - colocar o bit correspondente em `DDRx` com valor **0**.
4. **Definir se usará pull-up interno**:
   - escrever **1** em `PORTx` para **habilitar pull-up**;
   - escrever **0** em `PORTx` para deixar a entrada **sem pull-up**.
5. **Ler o estado do pino** no registrador `PINx`.

Exemplo:

Para o pino **PD0 sem PULL-UP**:

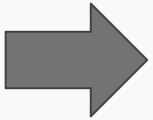
1. Selecionar o pino **PD0**.
2. Configurar **PD0 como entrada** em `DDRD`.
3. Escrever **0 em PORTD no bit 0** para manter o pino **sem pull-up**.
4. Ler o valor do pino em `PIND` no **bit 0**.

Exemplo 02 - Acionamento de um pino como entrada

Exemplo:

Para o pino **PD0 sem PULL-UP**:

1. Selecionar o pino **PD0**.
2. Configurar **PD0 como entrada** em `DDRD`.
3. Escrever **0 em PORTD no bit 0** para manter o pino **sem pull-up**.
4. Ler o valor do pino em `PIND` no **bit 0**.



Em C:

```
#include <avr/io.h>

int main(void)
{
        // Configura o pino PD0 como entrada (0)
        DDRD &= ~(1 << PD0);

        // Configura a entrada sem o PULL-UP
        PORTD &= ~(1 << PD0);

        // Desliga o pino PB5
        PORTB &= ~(1 << PB5);

        while (1)
        {
                // Verifica se pino PD0 está em 1
                if (PIND & (1 << PD0))
                        {
                                //nível alto
                        }
                else
                        {
                                //nível baixo
        }
}
```

Exemplo 02 - Verificação do pino PD0 como entrada, sem PULL-UP

Operações Bitwise utilizadas

(PIND & (1 << PD0))

Estou verificando se PD0 está setado

Nível alto: `if (PIND & (1 << PD0))`

Nível baixo: `if ((PIND & (1 << PD0)) == 0)`

De forma genérica

Para verificar se um bit está em nível alto

```
if (PINx & (1 << BIT))
        {
                // pino em nível alto
        }
```

Significado

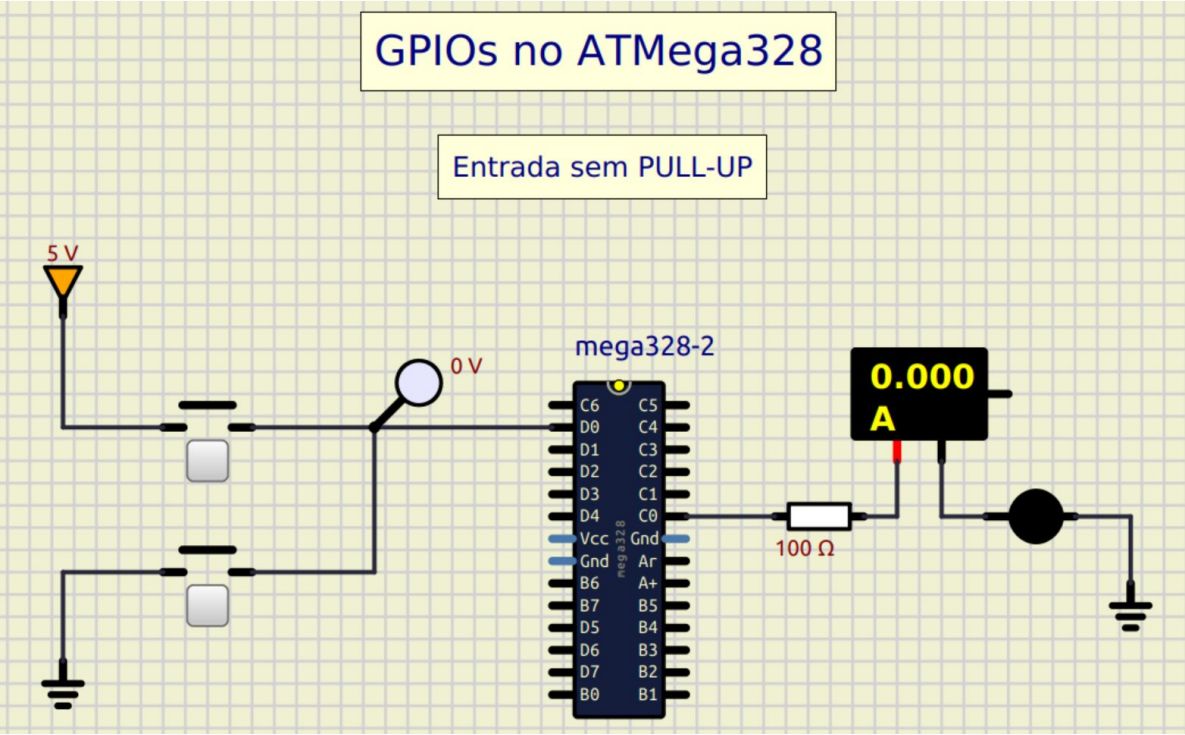
`1 << BIT` cria a máscara do bit do pino

`PINx & máscara` isola apenas esse bit

```
Se resultado = 0, bit não acionado
Se resultado <>0, bit acionado
```

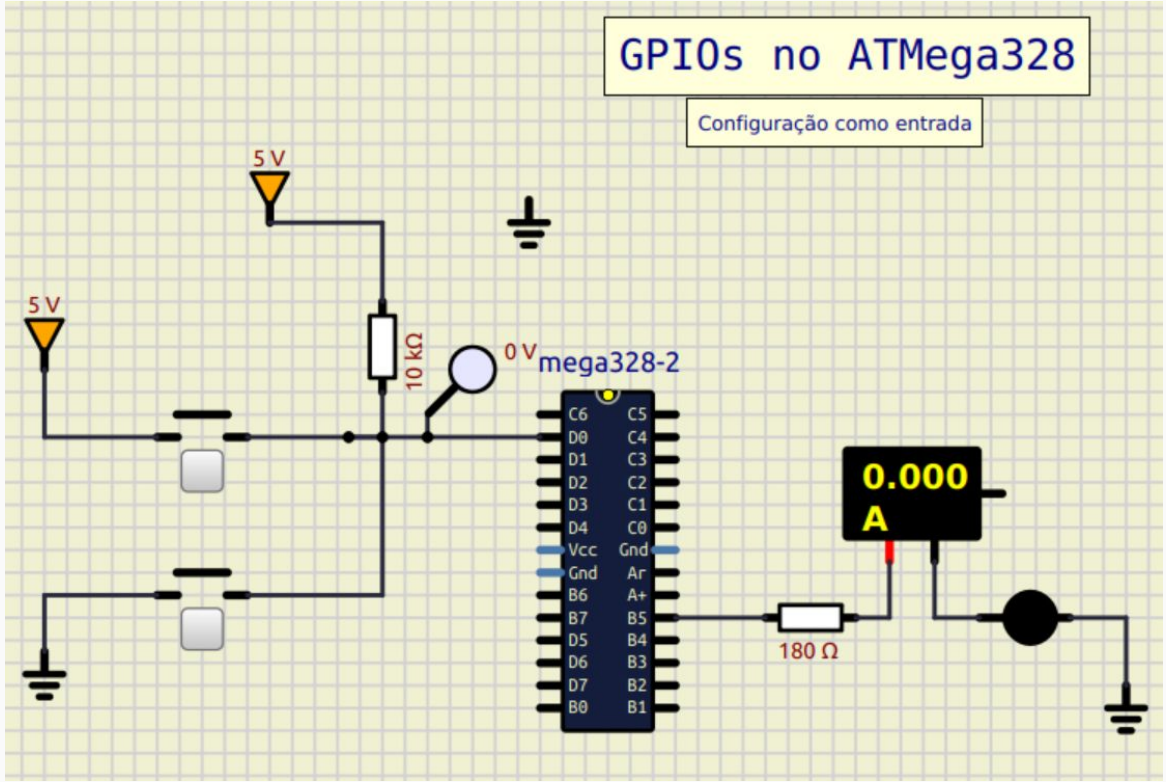
```
1 << 2 = 000000100 (máscara)

PIND & (1 << PD2)

Realiza a operação:
   PIND (estado atual dos pinos da portaD) com a
máscara

PIND      = 01101101 &
máscara   = 00000100
resultado = 00000100
Se, resultado diferente de zero - pino está setado
```

Simulação 2 - Entrada sem PULL-UP



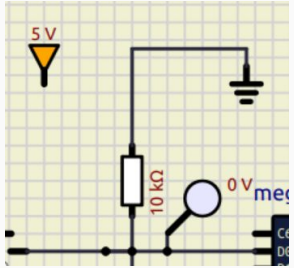

```c
#include <avr/io.h>

int main(void) {
    //Configura direções dos pinos

    DDRB |= (1 << PB5); //PB5 como saída (1)

    DDRD &= ~(1 << PD0);  //PD0 como entrada (0)

    //Configura PD0 SEM resistor de PULL-UP (0 em PD0 de PortD)

    DDRD &= ~(1 << PD0);

        while(1)
        {
        //Verifica se PD0 está em nível 1
        if (PIND & (1 << PD0))
            {
                PORTB |= (1 << PB5); //SETA PB5
            }
        else
            {
                PORTB &= ~(1<<PB5); //Reset de PB5
            }
        }
}
```

Simulação 3 - O que é Pull Up e Pull Down?



Resistor de Pull-up

- Evita a flutuação do pino quando não houver tensão
- Sinal fica em 5V quando o botão não está acionado e vai para 0V quando acionado

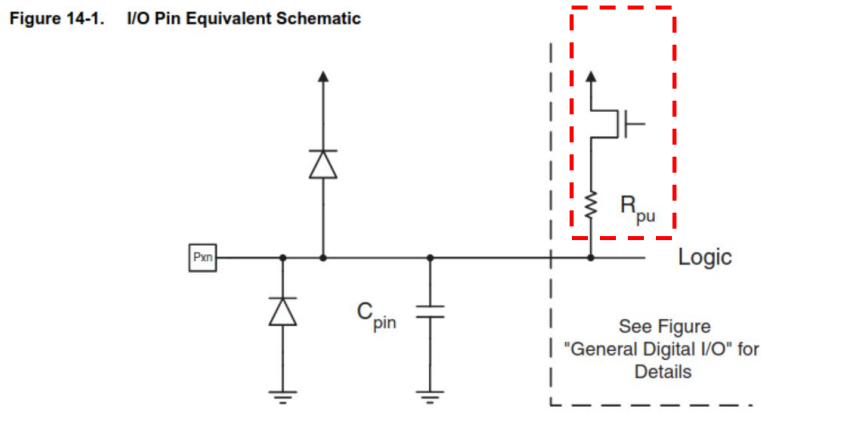


Resistor de Pull-down

- Evita a flutuação do pino quando não houver tensão
- Sinal fica em 0V quando o botão não está acionado e vai para 5V quando acionado

Simulação 3 - Entrada com PULL UP interno

ATMega possui resistor de **PULL-UP interno que pode ser habilitado por software**



Resistor de Pull-up interno

- Somente para saídas
- Para usar eu habilito o pino correspondente da PORT que estou usando
- Necessário **alterar a lógica do programa**

Exemplo 03 - Acionamento de um pino como entrada com PULL-UP interno

Exemplo:

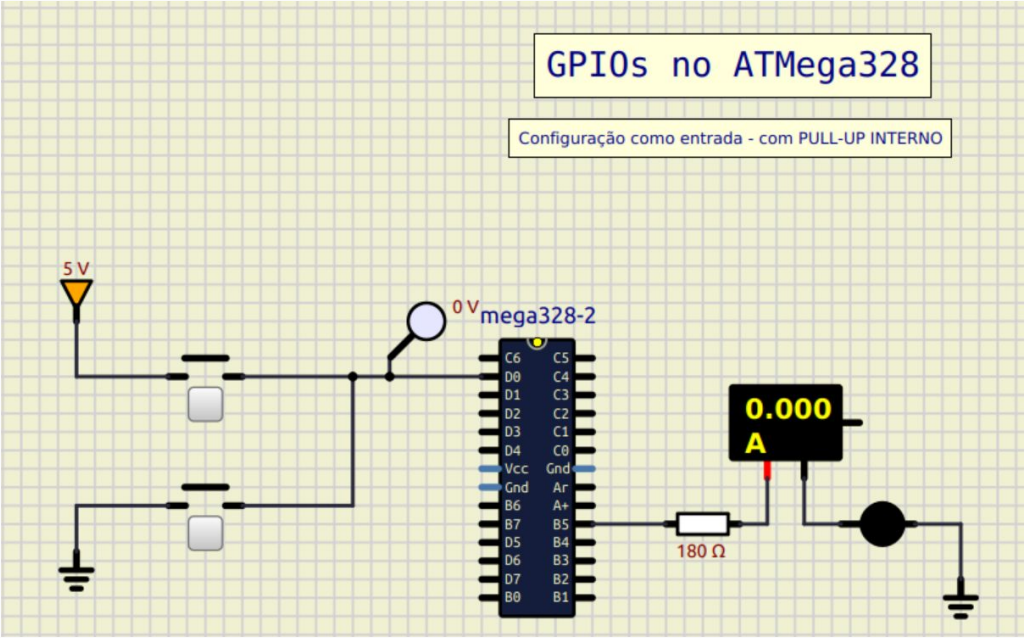
Para o pino **PD0 com PULL-UP**:

1. Selecionar o pino **PD0**.
2. Configurar **PD0 como entrada** em `DDRD`.
3. Escrever **1 em PORTD no bit 0** para manter o pino **sem pull-up**.
4. Ler o valor do pino em `PIND` no **bit 0**.

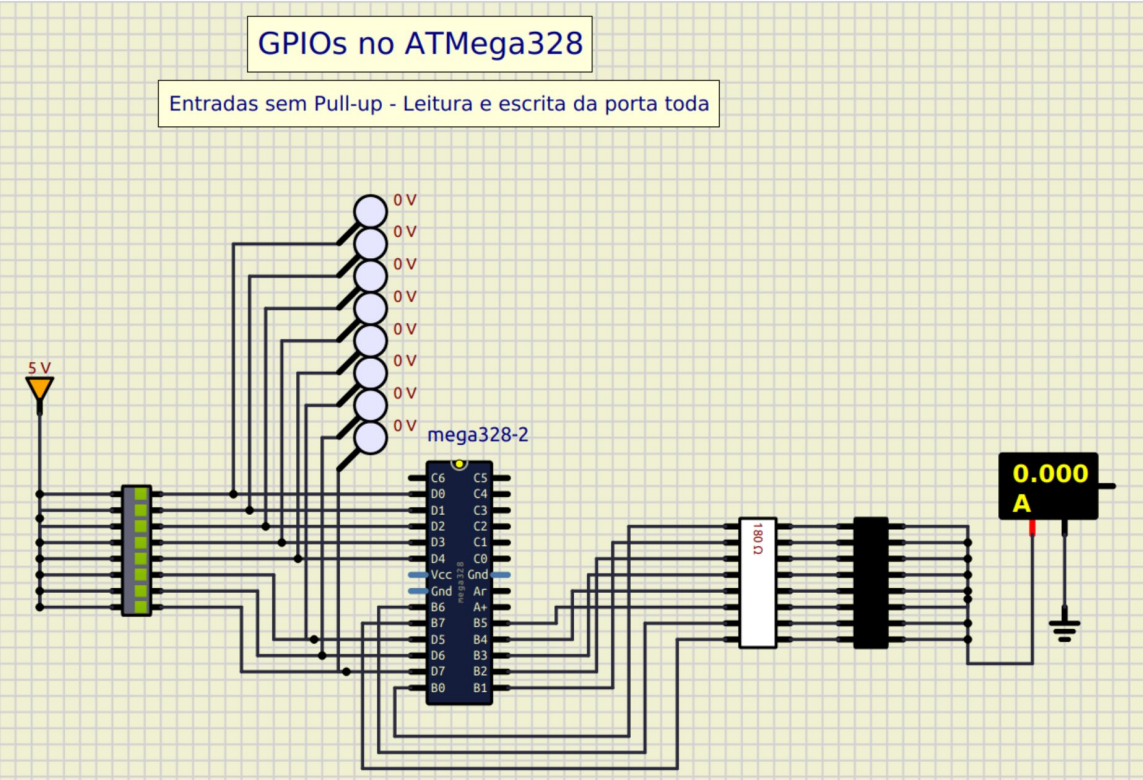
Em C:

```
#include <avr/io.h>

int main(void)
{
        // Configura o pino PD0 como entrada (0)
        DDRD &= ~(1 << PD0);

        // Configura a entrada com o PULL-UP
        PORTD |= (1 << PD0);

        // Desliga o pino PB5
        PORTB &= ~(1 << PB5);

        while (1)
        {
                // Verifica se pino PD0 está em 1
                if (PIND & (1 << PD0))
                        {
                                //nível alto
                        }
                else
                        {
                                //nível baixo
        }
}
```

Simulação 3 - Entrada com PULL-UP interno



Outras formas de acionamento dos pinos de I/O

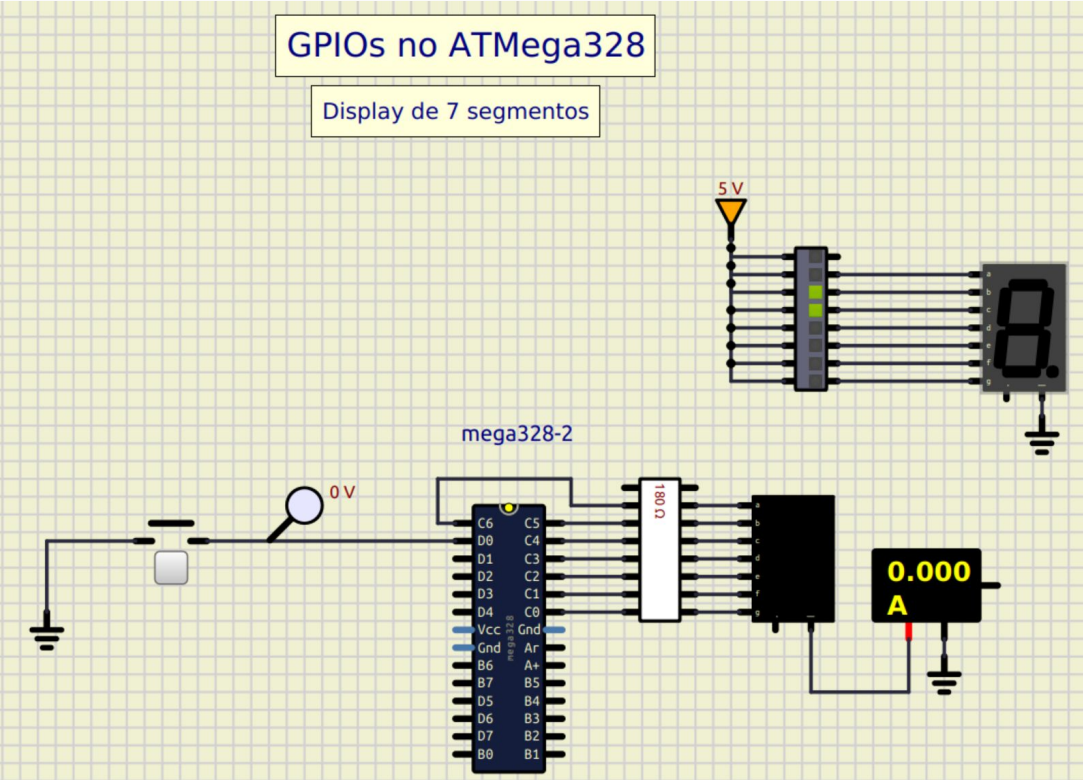


Leitura e escrita de todos os pinos

Simulação 04 - Lê portD e escreve em portB

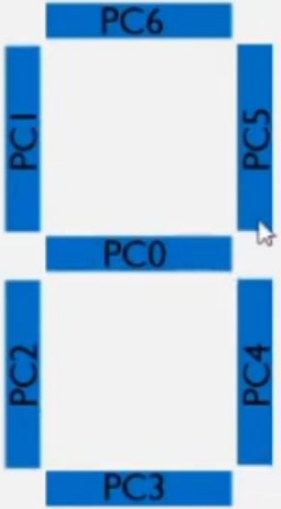
```c
#include <avr/io.h>

int main(void) {
    // Configuração dos GPIOs

    //PortB como saida (1) em todos os pinos
    DDRB = 0b11111111;

    //PORTD como entrada (0) em todos os pinos
    DDRD = 0b00000000;

    //Desabilitando o PULL-UP interno (0) em todos os pinos
    PORTD = 0b00000000;

    // Laço principal - Lê os dados em PIND e escreve em portB
    while (1)
    {
        PORTB = PIND;
    }
}
```

Simulação 05 - Display de 7 segmentos



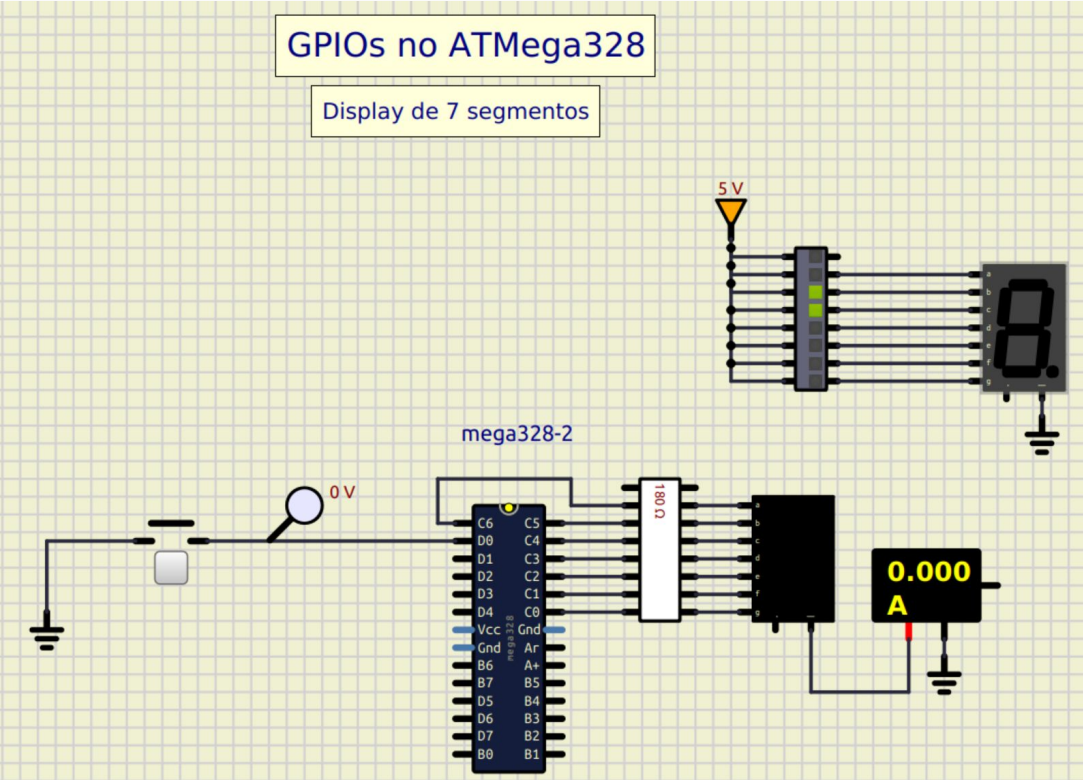
Simulação 05 - Display de 7 segmentos

- **Exemplo:** Contador com display de 7 segmentos

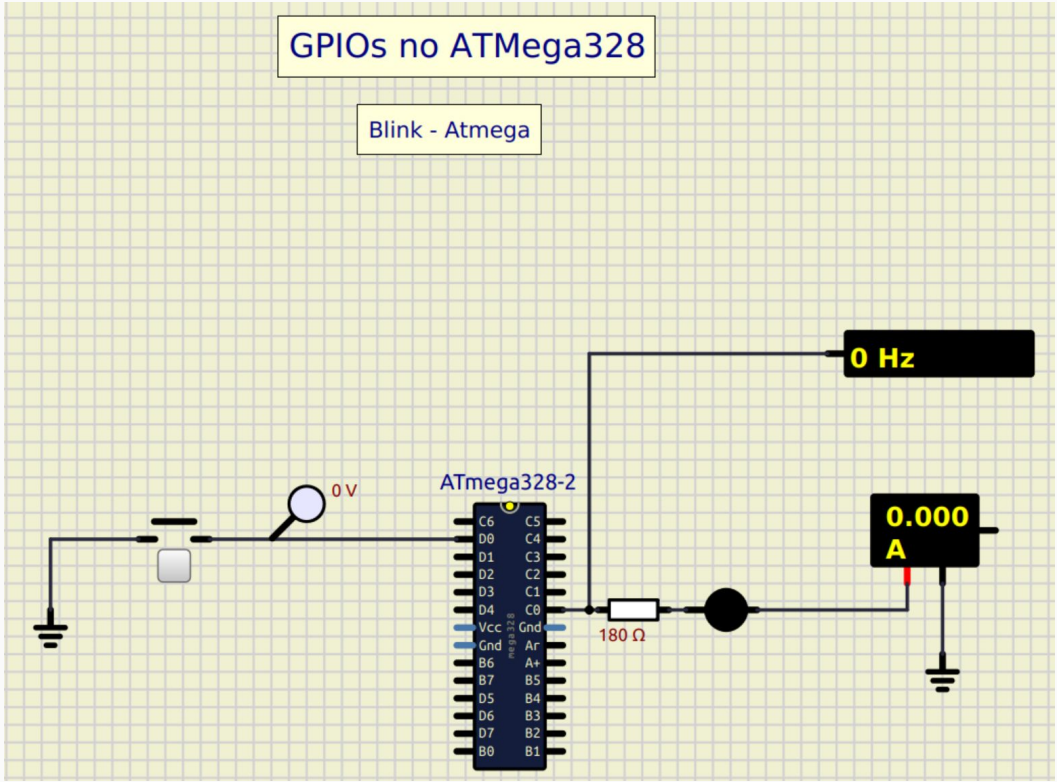


Nº	PC6	PC5	PC4	PC3	PC2	PC1	PC0	HEX
0	1	1	1	1	1	1	0	0x7E
1	0	1	1	0	0	0	0	0x30
2	1	1	0	1	1	0	1	0x6D
3	1	1	1	1	0	0	1	0x79
4	0	1	1	0	0	1	1	0x33
5	1	0	1	1	0	1	1	0x5B
6	1	0	1	1	1	1	1	0x5F
7	1	1	1	0	0	0	0	0x70
8	1	1	1	1	1	1	1	0x7F
9	1	1	1	1	0	1	1	0x7B

Simulação 05 - Display de 7 segmentos



Simulação 06 - Blink com ATmega - Uso do delay

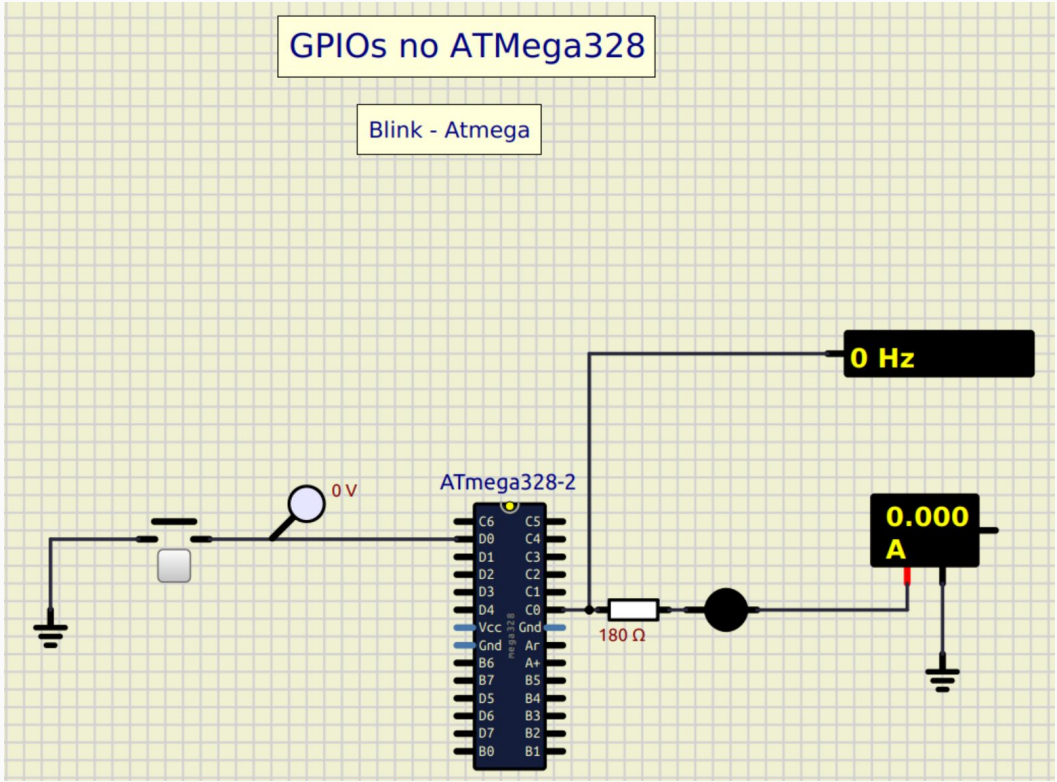


Para implementar o blink, precisaremos utilizar a biblioteca **util/delay.h do AVR** para utilizar a função delay(ms)

Será necessário informar também qual a frequência de clock do uC

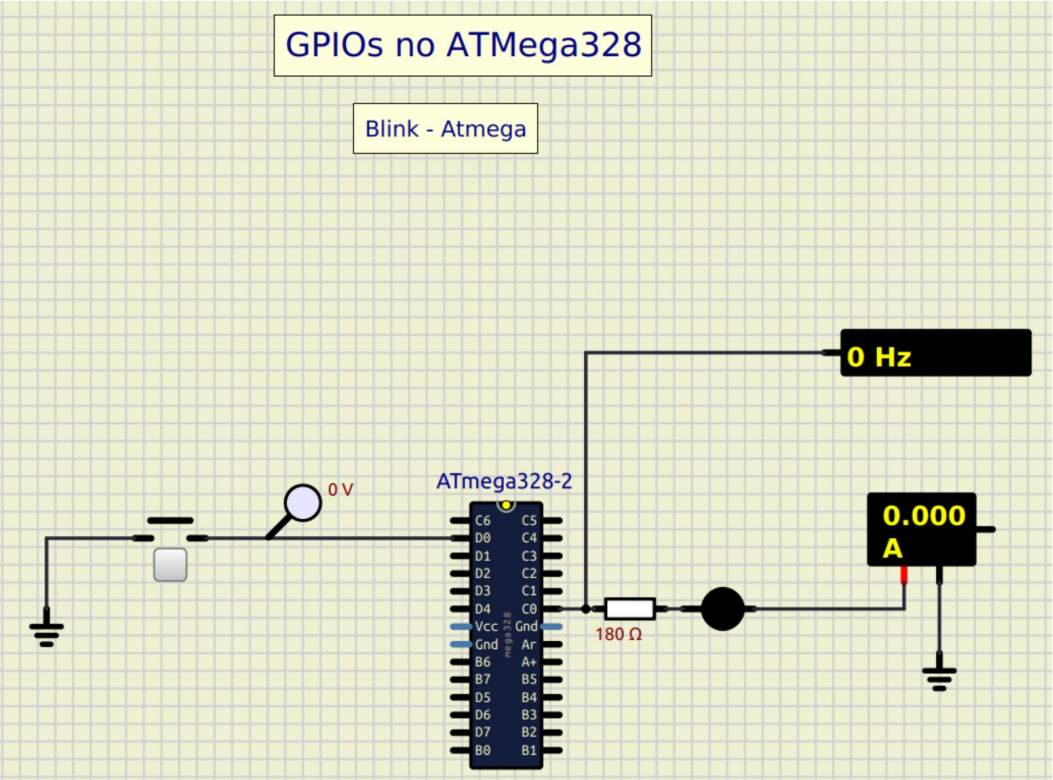
Importante: A frequência deve ser informada ANTES do carregamento da biblioteca util/delay.h

Simulação 06 - Blink com ATmega - Uso do delay



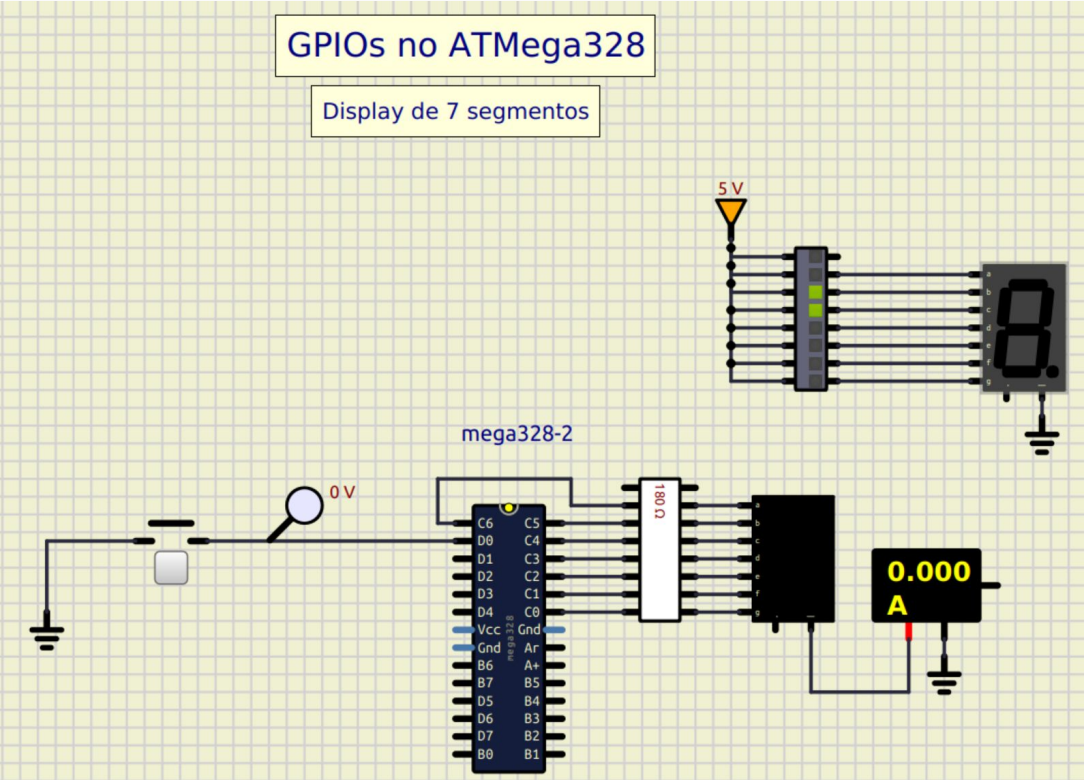

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>


int main(void)
{
DDRC |= (1 << PC0); // PC0 como saída

while (1)
        {
                PORTC |= (1 << PC0); // liga
                _delay_ms(500); // espera 500 ms

                PORTC &= ~(1 << PC0); // desliga
                _delay_ms(500); // espera 500 ms
        }
}
```

Simulação 07 - Se botão PD0 pressionado Blink




```c
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    // Configurações iniciais dos GPIOs
    DDRC |= (1 << PC0); //PC0 como saída (1)
    DDRD &= ~(1 << PD0); //PD0 como entrada (0)
    PORTD |= (1 << PD0); //Pull-up em PD0 (1)


    // Loop principal do blink
    while (1)
    {
        if ((PIND & (1 << PD0))== 0) //Se PD0 acionado
        {
            PORTC |= (1 << PC0); // PC0 = 1
            _delay_ms(200);      // Aguarda 200ms

            PORTC &= ~(1 << PC0); // PC0 = 0
            _delay_ms(200);      // Aguarda 200ms
        }
        else
        {
            PORTC &= ~(1 << PC0); // PC0 = 0
        }
    }
}
```

Simulação 08 - Contador 0 a 9 temporizado



Ao ser pressionado o botão em PD0 (com pull up) o contador deverá incrementar a contagem automaticamente a cada 500ms

Simulação 08 - Contador 0 a 9 temporizado

Ao ser pressionado o botão em PD0 (com pull up) o contador deverá incrementar a contagem automaticamente a cada 500ms

Algoritmo sem utilizar vetor:

- Configurar PD0 como entrada com Pull up
- Configurar PortC como saída
- Criar e inicializar variáveis:
  - numero_display - guarda valor exibido
  - tempo_delay - guarda valor de delay
- Repita continuamente:
- Verificar se botao esta acionado:
  - Se SIM;
    - mostre numero_display no LCD
    - aguarde tempo_delay
    - incremente numero_display
  - Se Não;
    - Não faça nada

Simulação 08 - Contador 0 a 9 temporizado

Algoritmo sem utilizar vetor:

- Configurar PD0 como entrada com Pull up
- Configurar PortC como saída
- Criar e inicializar variáveis:
  - numero_display - guarda valor exibido
  - tempo_delay - guarda valor de delay
- Repita continuamente:
- Verificar se botao esta acionado:
  - Se SIM;
    - mostre numero_display no LCD
    - aguarde tempo_delay
    - incremente numero_display
  - Se Não;
    - Não faça nada

```c
#define F_CPU 16000000UL

#include<util/delay.h>
#include <avr/io.h>

int main(void) {
    // 1. Configurações do GPIOs

    DDRD &= ~(1 << PD0); //PD0 como entrada (0)
    PORTD |= (1 << PD0); //PD0 com pull-up habilitado

    DDRC = 0b11111111;   //PORTC como saída (1) - LCD

    //Conexão PORTC = LCD
    // C0 - g; C1 - f; C2 - e
    // C3 - d; C4 - c; C5 - b
    // C6 - a;
    // codificação numeros:
    // 0 - 7E, 1 - 30, 2 - 6D, 3 -79  ,4 -33, 5 - 5B , 6 -5F , 7 - 70 , 8 - 7F , 9 - 7B ,

    // 2. Variaveis do programa
    int num_display = 0;    // Numero exibido no display
    int tempo_delay = 500;  // Tempo de delay em ms
```

Simulação 08 - Contador 0 a 9 temporizado

Algoritmo sem utilizar vetor:

- Configurar PD0 como entrada com Pull up
- Configurar PortC como saída
- Criar e inicializar variáveis:
  - numero_display - guarda valor exibido
  - tempo_delay - guarda valor de delay
- Repita continuamente:
- Verificar se botao esta acionado:
  - Se SIM;
    - Verifique qual o número
    - mostre numero_display no LCD
    - aguarde tempo_delay
    - incremente numero_display
    - Se número = 9, faça numero = 0
  - Se Não;
    - Não faça nada

```
while (1)

{
    if ((PIND & (1 << PD0))== 0)     //Se PD0 acionado (==0)
    {
        if (num_display == 0)
        {
            PORTC = 0x7E;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 1)
        {
            PORTC = 0x30;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 2)
        {
            PORTC = 0x6D;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 3)
        {
            PORTC = 0x79;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 4)
        {
            PORTC = 0x33;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 5)
        {
```

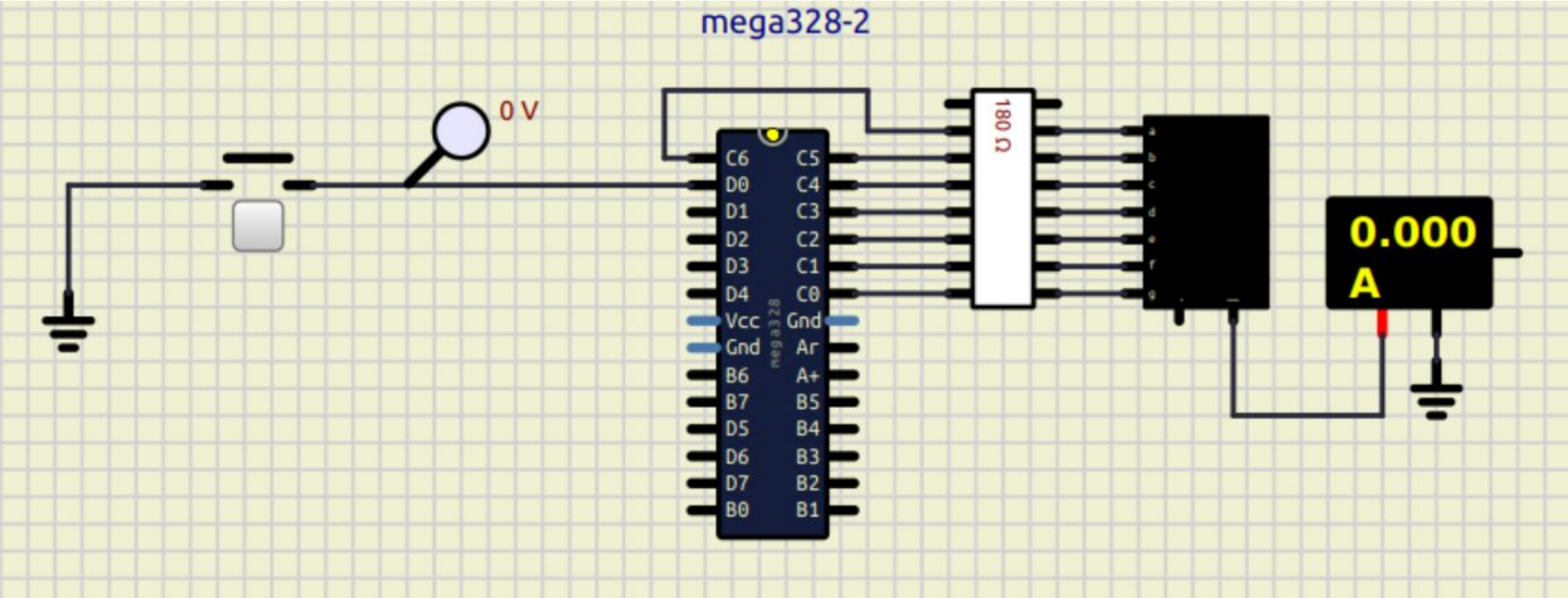
Simulação 08 - Contador 0 a 9 temporizado

Algoritmo sem utilizar vetor:

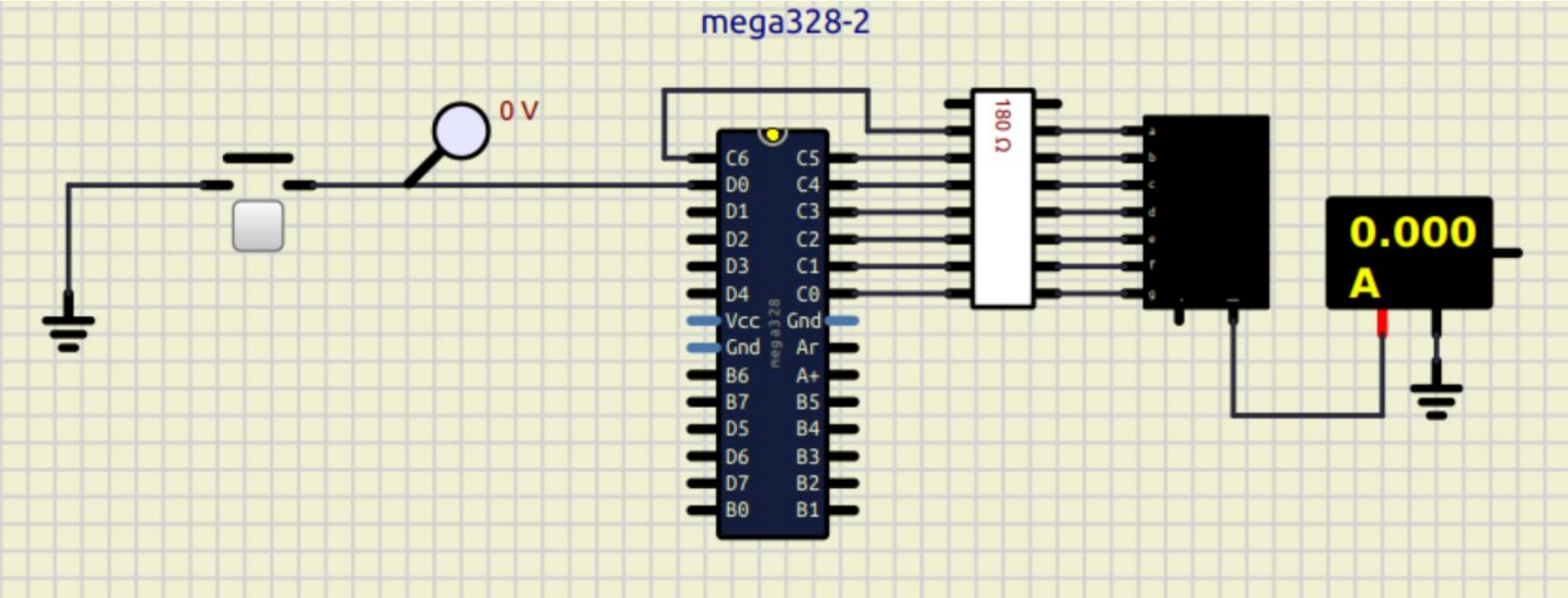
- Configurar PD0 como entrada com Pull up
- Configurar PortC como saída
- Criar e inicializar variáveis:
  - numero_display - guarda valor exibido
  - tempo_delay - guarda valor de delay
- Repita continuamente:
- Verificar se botao esta acionado:
  - Se SIM;
    - Verifique qual o número
    - mostre numero_display no LCD
    - aguarde tempo_delay
    - incremente numero_display
    - Se número = 9, faça numero = 0
  - Se Não;
    - Não faça nada

```
        else if (num_display == 4)
        {
            PORTC = 0x33;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 5)
        {
            PORTC = 0x5B;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 6)
        {
            PORTC = 0x5F;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 7)
        {
            PORTC = 0x70;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 8)
        {
            PORTC = 0x7F;
            _delay_ms(tempo_delay);
            num_display++;
        }
        else if (num_display == 9)
        {
            PORTC = 0x7B;
            _delay_ms(tempo_delay);
            num_display = 0; // reinicia a contagem
        }

    }
```

Simulação 08 - Contador 0 a 9 temporizado



Simulação 09 - Contador 0 a 9 temporizado - usando Vetor



Simulação 09 - Contador 0 a 9 temporizado - usando vetor

Ao ser pressionado o botão em PD0 (com pull up) o contador deverá incrementar a contagem automaticamente a cada 500ms

Algoritmo sem utilizar vetor:

- Configurar PD0 como entrada com Pull up
- Configurar PortC como saída
- Criar e inicializar variáveis:
  - numero_display - guarda valor exibido
  - tempo_delay - guarda valor de delay
  - **vetor_LCD - guarda os valores codificados dos números para exibição no display**
- Repita continuamente:
- Verificar se botao esta acionado:
  - Se SIM;
    - mostre numero_display no LCD
    - aguarde tempo_delay
    - incremente numero_display
  - Se Não;
    - Não faça nada

Simulação 09 - Contador 0 a 9 temporizado - usando vetor

Algoritmo sem utilizar vetor:

- Configurar PD0 como entrada com Pull up
- Configurar PortC como saída
- Criar e inicializar variáveis:
  - numero_display - guarda valor exibido
  - tempo_delay - guarda valor de delay
  - **vetor_LCD - guarda os valores codificados dos números para exibição no display**
- Repita continuamente:
- Verificar se botao esta acionado:
  - Se SIM;
    - mostre numero_display no LCD
    - aguarde tempo_delay
    - incremente numero_display
  - Se Não;
    - Não faça nada

```c
int main(void) {

    // 1. Configurações do GPIOs

    DDRD &= ~(1 << PD0); //PD0 como entrada (0)
    PORTD |= (1 << PD0); //PD0 com pull-up habilitado

    DDRC = 0b11111111;   //PORTC como saída (1) - LCD

    //Conexão PORTC = LCD
    // C0 - g; C1 - f; C2 - e
    // C3 - d; C4 - c; C5 - b
    // C6 - a;
    // codificação numeros:
    // 0 - 7E, 1 - 30, 2 - 6D, 3 -79  ,4 -33, 5 - 5B , 6 -5F , 7 - 70 , 8 - 7F , 9 - 7B ,

    // 2. Variaveis do programa
    int num_display = 0;    // Numero exibido no display
    int tempo_delay = 200;  // Tempo de delay em ms
    int vetor_LCD [10] = {0x7E,0x30,0x6D,0x79,0x33,0x5B,0x5F,0x70,0x7F,0x7B};

    // 3. Repetir continuamente
    while (1)
    {
    if ((PIND & (1 << PD0))== 0)    //Se PD0 acionado (==0)
        {
            PORTC = vetor_LCD[num_display]; //Exibir numero no LCD
            _delay_ms(tempo_delay); //Aguarda XX ms
            num_display++;  //incrementa numero
            if (num_display > 9) //Verifica se chegou no valor máximo
            {
                num_display = 0;
            }
        }
    }
}
```

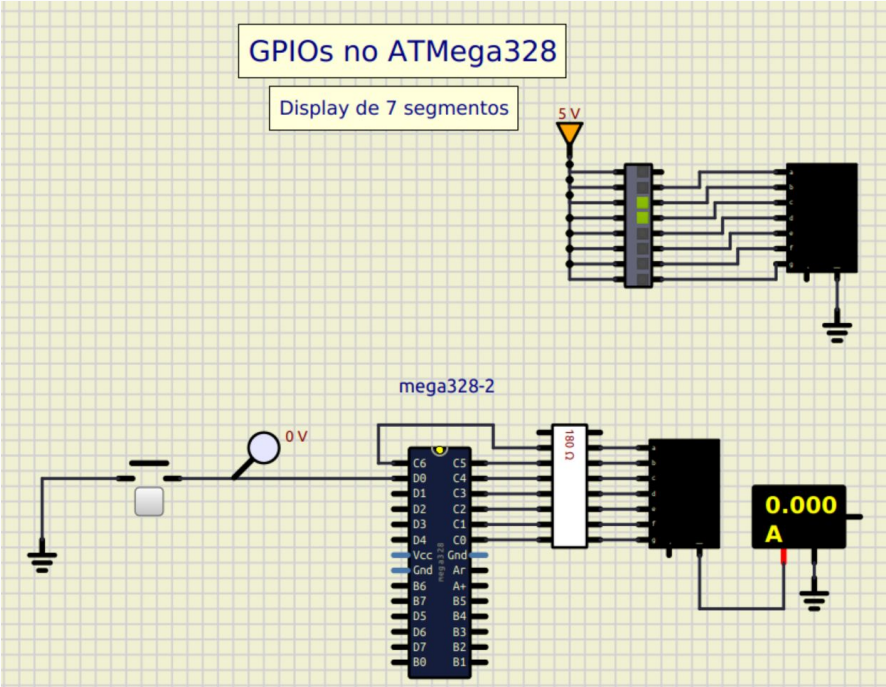
Simulação 09 - Contador 0 a 9 temporizado - usando vetor

Algoritmo sem utilizar vetor:

- Configurar PD0 como entrada com Pull up
- Configurar PortC como saída
- Criar e inicializar variáveis:
  - numero_display - guarda valor exibido
  - tempo_delay - guarda valor de delay
  - **vetor_LCD - guarda os valores codificados dos números para exibição no display**
- Repita continuamente:
- Verificar se botao esta acionado:
  - Se SIM;
    - mostre numero_display no LCD
    - aguarde tempo_delay
    - incremente numero_display
  - Se Não;
    - Não faça nada

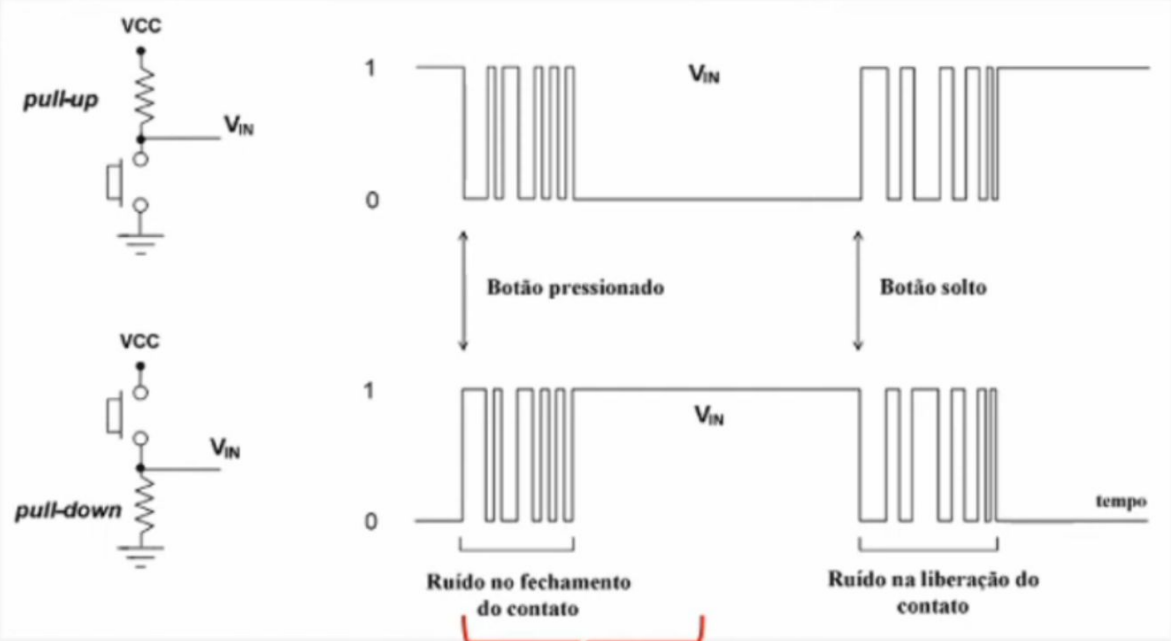
```c
int main(void) {

    // 1. Configurações do GPIOs

    DDRD &= ~(1 << PD0); //PD0 como entrada (0)
    PORTD |= (1 << PD0); //PD0 com pull-up habilitado

    DDRC = 0b11111111;   //PORTC como saída (1) - LCD

    //Conexão PORTC = LCD
    // C0 - g; C1 - f; C2 - e
    // C3 - d; C4 - c; C5 - b
    // C6 - a;
    // codificação numeros:
    // 0 - 7E, 1 - 30, 2 - 6D, 3 -79  ,4 -33, 5 - 5B , 6 -5F , 7 - 70 , 8 - 7F , 9 - 7B ,

    // 2. Variaveis do programa
    int num_display = 0;    // Numero exibido no display
    int tempo_delay = 200;  // Tempo de delay em ms
    int vetor_LCD [10] = {0x7E,0x30,0x6D,0x79,0x33,0x5B,0x5F,0x70,0x7F,0x7B};

    // 3. Repetir continuamente
    while (1)
    {
    if ((PIND & (1 << PD0))== 0)    //Se PD0 acionado (==0)
        {
            PORTC = vetor_LCD[num_display]; //Exibir numero no LCD
            _delay_ms(tempo_delay); //Aguarda XX ms
            num_display++;  //incrementa numero
            if (num_display > 9) //Verifica se chegou no valor máximo
            {
                num_display = 0;
            }
        }
    }
}
```

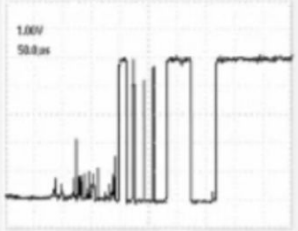
Simulação 09 - Contador 0 a 9 temporizado - usando vetor



Simulação 10 - Debouncing

- **Bounce:** Existe um ruído ao pressionar qualquer tipo de botão mecânico





Ruído em um botão real

Solução simplista: Adicionar um **delay**

Simulação 10 - Debouncing

Algoritmo debouncing de botão (solução simplificada)

1. Configurar o botão como **entrada**.
2. Habilitar o **pull-up interno**, se necessário.
3. Configurar a saída desejada.

Repetir

4. Verificar se o botão foi pressionado.
5. Aguardar um pequeno tempo para **debouncing**.
6. Confirmar o pressionamento com uma nova leitura.
7. Executar a ação desejada.
8. Esperar o botão ser solto.
9. Aguardar novamente um pequeno tempo para **debouncing da soltura**.
10. Retornar ao início do laço.

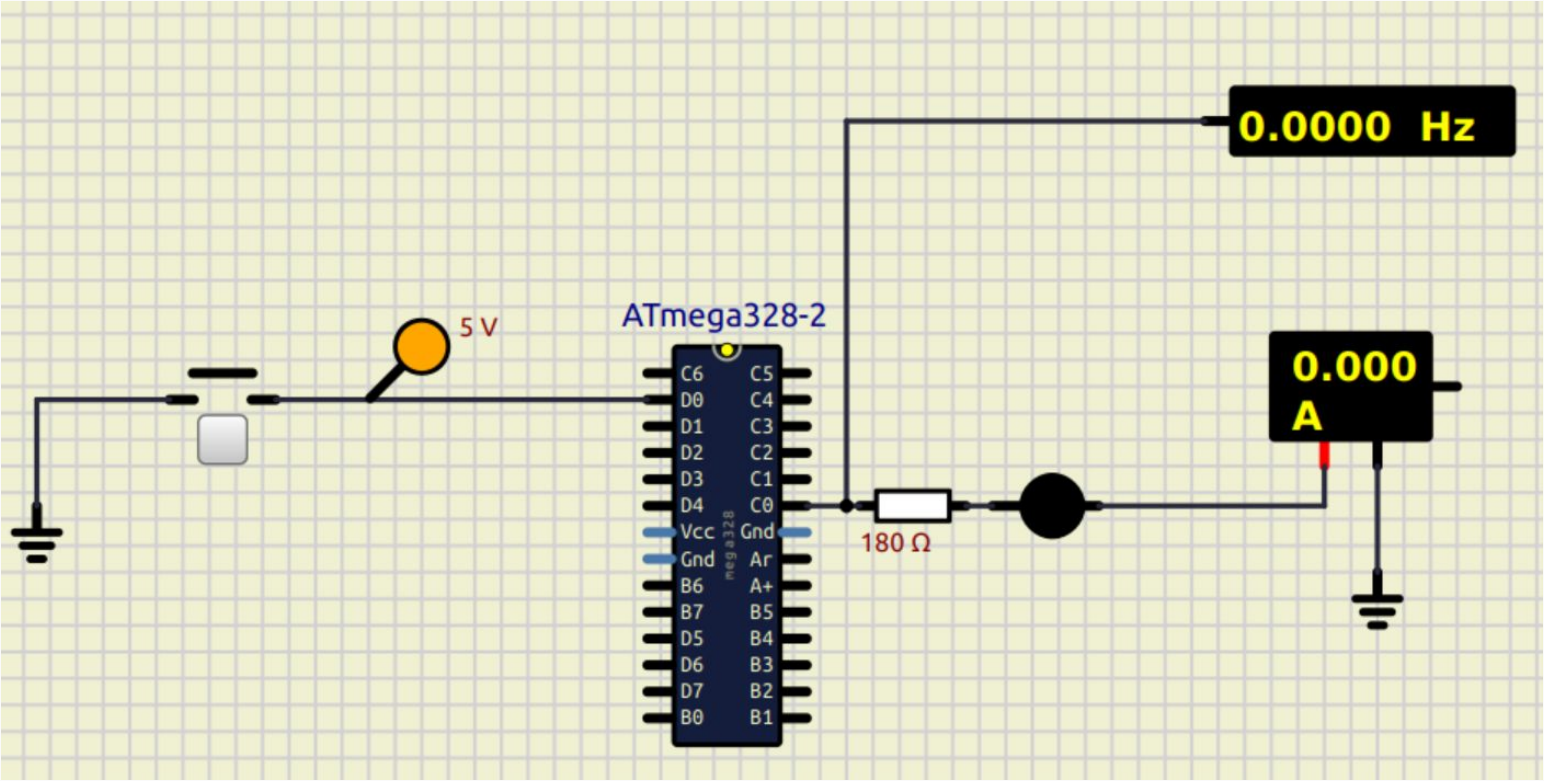
Simulação 10 - Debouncing

Algoritmo aplicado a PD0 e PC0

1. Configurar `PD0` como **entrada**
2. Habilitar o **pull-up interno** em `PD0`
3. Configurar `PC0` como **saída**
4. Repetir continuamente
    1. Verificar se o botão foi pressionado.
    2. Se o botão aparenta estar pressionado, aguardar **20 ms**.
    3. Ler novamente o botão para confirmar o pressionamento.
    4. Se o pressionamento for válido:
        1. Acender o LED.
        2. Permanecer nesse estado enquanto o botão estiver pressionado.
        3. Quando o botão for solto, aguardar **20 ms** para debouncing da soltura.
        4. Apagar o LED.
5. Voltar ao início do laço.

```c
#include <util/delay.h>
#include <avr/io.h>

int main(void)
{
    // 1. Configurar PD0 como entrada
    // 2. Habilitar o pull-up interno em PD0
    // 3. Configurar PC0 como saída

    DDRD &= ~(1 << PD0);
    PORTD |= (1 << PD0);
    DDRC |= (1 << PC0);

    // 4. Repetir continuamente
    while (1)
    {
        // 4.1 Verificar se o botão aparenta estar pressionado (==0)
        if ((PIND & (1 << PD0)) == 0)
        {
            // 4.2 Aguardar 20 ms para estabilizar o sinal
            _delay_ms(20);

            // 4.3 Ler novamente para confirmar o pressionamento
            if ((PIND & (1 << PD0)) == 0)
            {
                // 4.4.1 Acender o LED
                PORTC |= (1 << PC0);

                // 4.4.2 Permanecer aqui enquanto o botão estiver pressionado
                while ((PIND & (1 << PD0)) == 0)
                {
                }

                // 4.4.3 Aguardar 20 ms para debouncing da soltura
                _delay_ms(20);

                // 4.4.4 Apagar o LED
                PORTC &= ~(1 << PC0);
            }
        }
    }
}
```

Simulação 10 - Debouncing

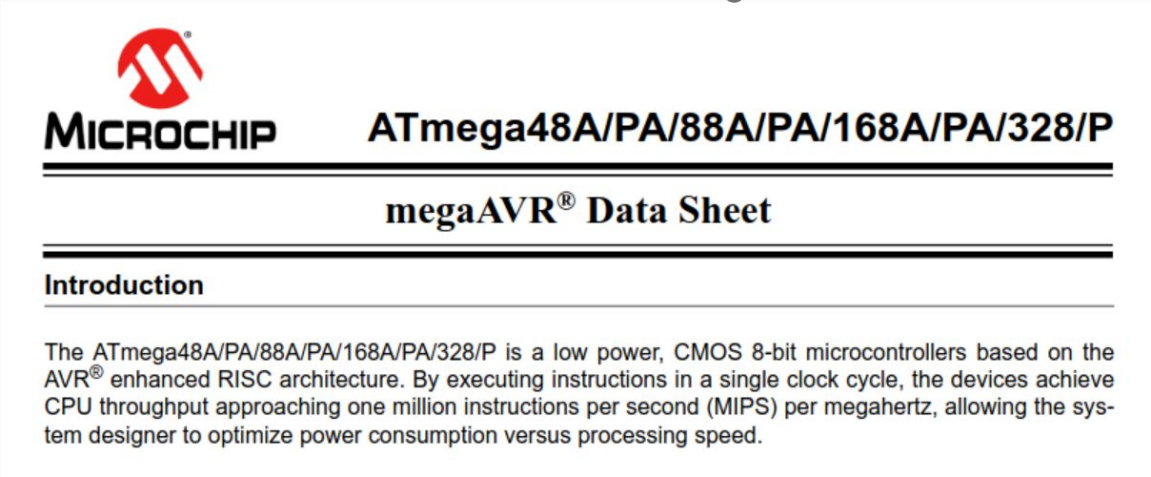


Materiais de referência:

Vídeo-aulas

- GPIOs

Data sheet - ATmega328



MICROCHIP

ATmega48A/PA/88A/PA/168A/PA/328/P

megaAVR® Data Sheet

Introduction

The ATmega48A/PA/88A/PA/168A/PA/328/P is a low power, CMOS 8-bit microcontrollers based on the AVR® enhanced RISC architecture. By executing instructions in a single clock cycle, the devices achieve CPU throughput approaching one million instructions per second (MIPS) per megahertz, allowing the system designer to optimize power consumption versus processing speed.

Livro

Livro - AVR - Técnicas de projeto